Event Tracking System Architecture

The goal is to build an event tracking platform that can:

- Ingest millions of events per day
- Process and store data within minutes
- Support historical analytics
- Scale when traffic grows
- Minimize infrastructure cost
- Minimize operational and maintenance complexity

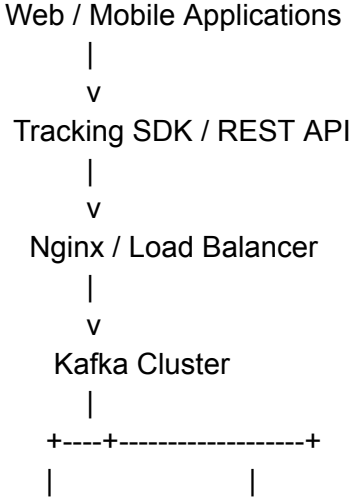
Because cost efficiency is the main priority, I would evaluate three architectures:

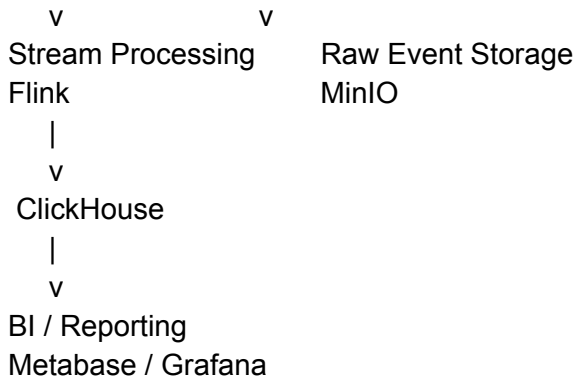
1. Self-Hosted / On-Premise on VPS
2. Fully Managed Cloud
3. Hybrid Architecture

My preferred solution would be the **Hybrid Architecture**, because it provides a good balance between low infrastructure finance cost and low maintenance overhead.

Solution 1: Self-Hosted / On-Premise on VPS

High-Level Architecture





Technical Stack

Ingestion

A stateless event tracking API can be built using FastAPI, Go, or Node.js and deployed behind Nginx.

Example event:

```
{
  "event_id": "uuid",
  "user_id": 12345,
  "event_type": "product_view",
  "product_id": 1001,
  "event_time": "2026-10-05T12:30:00Z",
  "platform": "web"
}
```

The API should perform only lightweight validation and immediately publish events to the streaming layer.

Streaming - Apache Kafka.

Kafka provides:

- High throughput
- Message durability
- Consumer groups
- Partition-based scalability
- Replay capability

Stream Processing

Apache Flink would consume events from Kafka and perform:

- Schema validation
- Deduplication
- Transformation
- Enrichment
- Invalid event routing

```
Kafka
  |
  v
Flink
  |
  +--> Valid Events --> ClickHouse
  |
  +--> Raw Events --> MinIO
  |
  +--> Invalid Events --> DLQ
```

Analytical Storage - ClickHouse

It is a good fit for event workloads because it offers:

- High query performance
- Columnar storage
- Strong compression
- Efficient aggregations

Historical Storage - MinIO

Events would be stored in Parquet format:

```
events/
  year=2026/
    month=10/
      day=05/
        hour=12/
          events.parquet
```

Recent data could remain in ClickHouse while older data is kept only in MinIO.

Observability

- Prometheus
- Grafana

- Loki
- Alertmanager

Important metrics include:

- Event ingestion rate
- Kafka consumer lag
- Flink processing latency
- DLQ volume
- ClickHouse ingestion delay
- CPU / RAM / disk usage
- Data freshness

Cost Characteristics

The biggest advantage is low infrastructure cost.

A few VPS instances can run the system much more cheaply than several managed cloud services.

However, the hidden cost is operational complexity.

The team must maintain:

- Kafka
- Flink
- ClickHouse
- MinIO
- Backups
- Replication
- Monitoring
- Failover
- OS updates
- Security patches

Therefore:

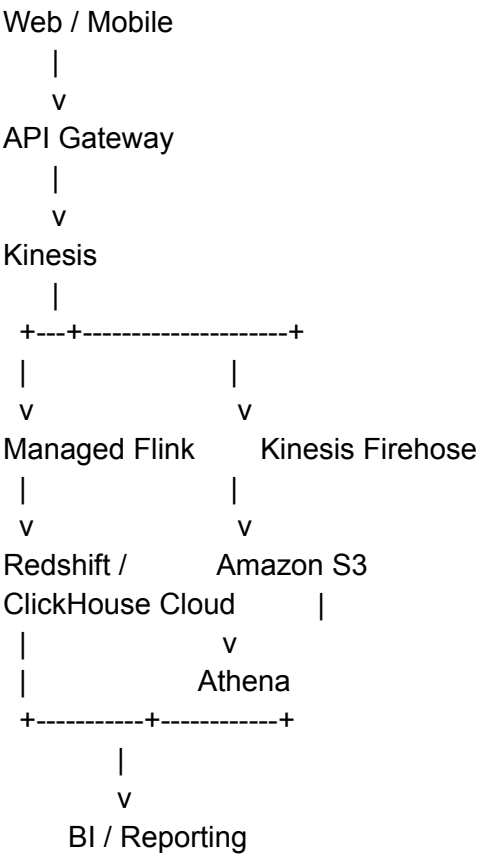
+ Infrastructure cost: LOW
+ Maintenance cost: HIGH
+ Operational complexity: HIGH

This architecture is most suitable when the company already has strong infrastructure or DevOps expertise.

Solution 2: Fully Managed Cloud

For this design, AWS is used as an example.

High-Level Architecture



Technical Stack

Ingestion

Amazon API Gateway provides:

- Automatic scaling
- TLS
- Rate limiting
- Authentication
- Routing

Streaming - Amazon Kinesis

If Kafka compatibility is required, Amazon MSK is another option.

Processing

For simple processing: AWS Lambda

For stateful streaming: Amazon Managed Service for Apache Flink

Historical Storage - Amazon S3

Events should be stored as compressed Parquet files.

Historical Query - Amazon Athena

Near Real-Time Analytics

Possible options include:

- Amazon Redshift
- ClickHouse Cloud
- Snowflake

Observability

CloudWatch can monitor:

- Error rates
- Kinesis lag
- Lambda failures
- Flink processing latency
- DLQ count
- Storage and warehouse metrics

Cost Characteristics

The cloud architecture minimizes operational work because most components are managed.

However, managed services can become expensive.

Typical cost drivers include:

- API Gateway requests
- Kinesis capacity
- Managed Flink
- Redshift compute

- Data transfer
- CloudWatch logs
- Storage requests

Therefore:

+ Infrastructure cost: MEDIUM to HIGH
+ Maintenance cost: LOW
+ Operational complexity: LOW

This option is attractive when engineering time is more expensive than infrastructure.

Solution 3: Hybrid Architecture

For the given requirements, I would prefer a hybrid architecture.

The idea is simple:

> Self-host the components that are cheap and easy to maintain, while using cloud services only where they provide strong operational value.

The architecture should avoid both extremes:

Everything self-hosted
↓
Cheap infrastructure
but expensive operations

and:

Everything managed
↓
Simple operations
but potentially expensive infrastructure

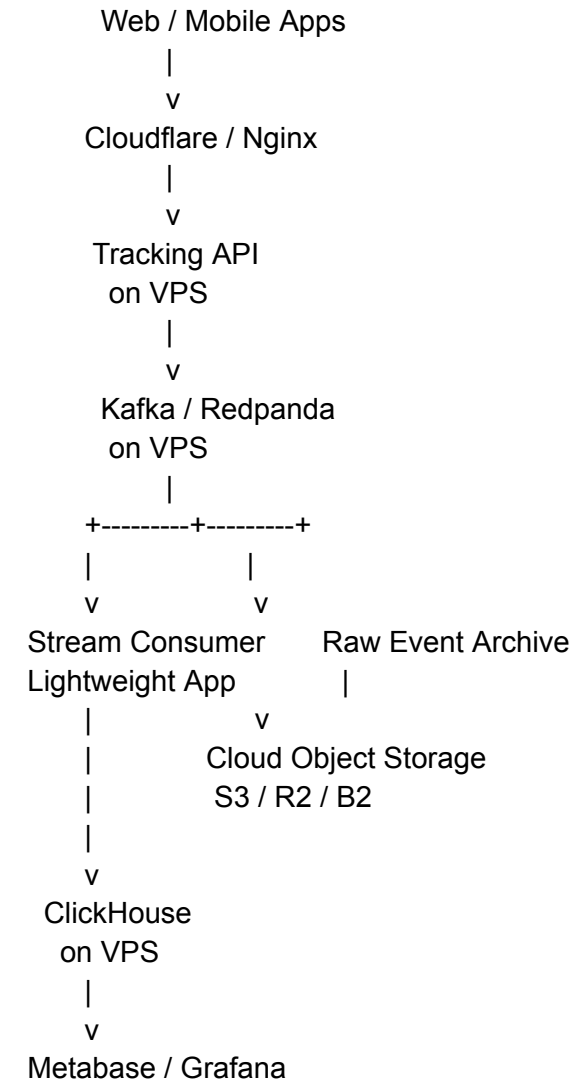
Instead:

Keep the simple workloads on inexpensive VPS infrastructure.

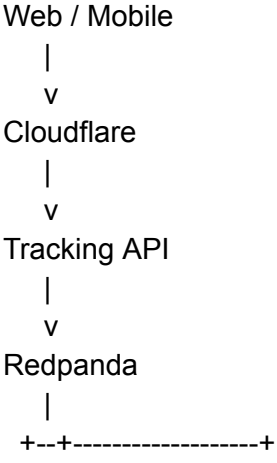
Use low-cost cloud object storage for durable historical data.

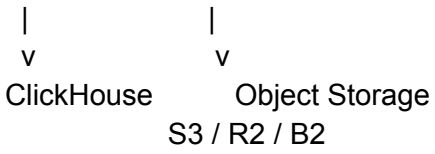
Avoid expensive managed streaming and warehouse services unless they are necessary.

Proposed Hybrid Architecture



A more cost-optimized version could use:





This reduces the number of components significantly.

Hybrid Technical Stack

1. Edge Layer

I would use Cloudflare in front of the tracking API.

Cloudflare provides:

- DNS
- TLS
- Basic DDoS protection
- Rate limiting
- CDN infrastructure

This reduces the amount of infrastructure the team needs to operate.

---

2. Tracking API on VPS

The tracking API could run as multiple Docker containers.

Possible stack:

Go / FastAPI
+
Docker
+
Nginx

I would prefer Go for a very lightweight ingestion service because it can handle high concurrency with relatively low CPU and RAM usage.

However, FastAPI is also perfectly reasonable when development speed matters more.

---

3. Kafka vs Redpanda

Redpanda is Kafka API compatible but is simpler to operate.

Traditional Kafka may require:

+ Kafka brokers
+ KRaft configuration
+ JVM tuning
+ More operational knowledge

Redpanda can provide a similar event streaming model with fewer operational components.

For this use case, Redpanda can reduce maintenance complexity while still providing:

- Partitioning
- Retention
- Consumer groups
- Replay
- Kafka API compatibility

---

4. Avoid Flink Initially

One important cost optimization is that I would **not introduce Flink unless the business actually needs complex streaming computation**.

For the current requirements, processing mainly involves:

Validation
Deduplication
Basic transformations
Enrichment
Routing

These operations can often be handled by a normal streaming consumer.

For example:

Redpanda

```
   |
   v
Go / Python Consumer
   |
   +--> ClickHouse
   |
   +--> Object Storage
   |
   +--> DLQ
```

This is significantly easier to maintain than:

```
Kafka
+
Flink Cluster
+
Checkpointing
+
Job Manager
+
Task Managers
```

If requirements later become more complex, such as:

- Sessionization
- Complex event processing
- Large streaming joins
- Stateful window aggregations

then Flink can be introduced.

The principle is:

> Do not introduce distributed systems before the workload requires them.

---

5. ClickHouse on VPS

ClickHouse would remain self-hosted because it provides excellent analytical performance even on relatively inexpensive servers.

For example:

Recent 30–90 days
|
v
ClickHouse

ClickHouse supports compression extremely well, which helps reduce both disk usage and infrastructure cost.

For millions of daily events, a reasonably sized VPS can often handle the workload comfortably.

---

6. Cloud Object Storage for Historical Data

I would not self-host long-term object storage unless required.

Instead, I would use inexpensive cloud object storage.

Possible options include:

Amazon S3
Cloudflare R2
Backblaze B2
Wasabi

The exact provider depends on company requirements.

Object storage is an area where managed cloud services provide excellent value because the company does not need to manage:

- Disk failures
- RAID
- Replication
- Storage capacity expansion
- Backup infrastructure
- Hardware replacement

Historical events would be stored as compressed Parquet.

Object Storage
|
+-- year=2026/
+-- month=10/
+-- day=05/

This provides cheap and durable long-term storage with almost no operational overhead.

---

7. Historical Analytics

Historical data does not necessarily need to be loaded back into ClickHouse.

For infrequent historical analysis, tools such as:

DuckDB
Trino
Athena

can query Parquet files directly.

For example, an analyst could query:

```
SELECT
   event_type,
   COUNT(*)
FROM historical_events
WHERE event_date BETWEEN '2026-01-01' AND '2026-01-31'
GROUP BY event_type;
```

This avoids keeping many years of data in expensive high-performance storage.

---

8. Hot and Cold Storage

The hybrid design uses a clear hot/cold storage strategy.

HOT DATA

ClickHouse

Recent 30–90 days
Fast dashboards
Frequent analytics
COLD DATA

Cloud Object Storage

Parquet

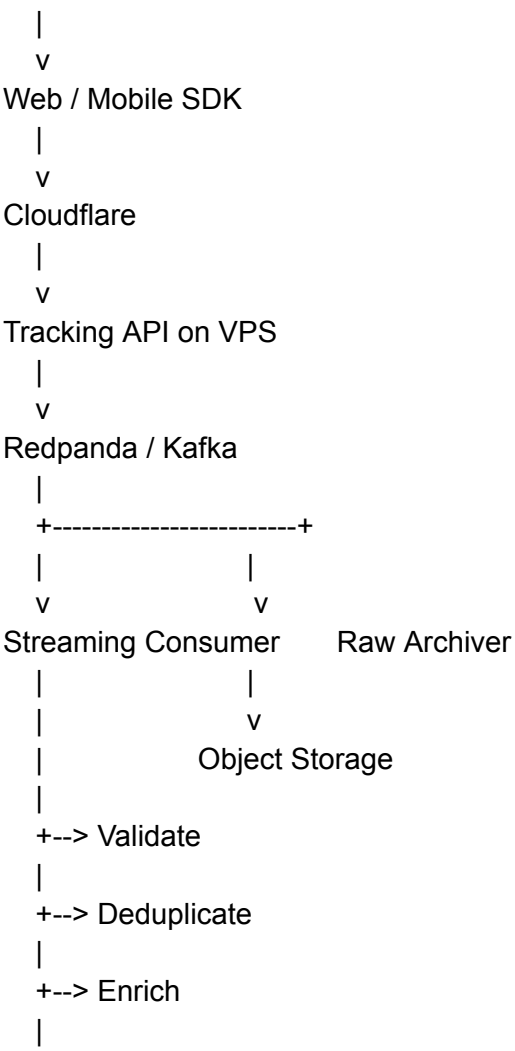
Older events
Historical analysis
Recovery
Reprocessing

This is one of the biggest cost optimizations in the architecture.

---

9. Data Flow

The complete hybrid data flow would be:



+--> DLQ invalid events
|
v
ClickHouse
|
v
Metabase / Grafana

---

10. Data Quality

Data quality should be implemented close to the ingestion pipeline.

Each event should have:

event_id
event_type
event_time
schema_version
platform

The consumer validates:

- Required fields
- Data types
- Timestamp validity
- Allowed event types
- Allowed platforms
- Duplicate event IDs

For schema management, I would avoid adding Schema Registry initially unless multiple teams and many event schemas require it.

An initial version could use:

JSON Schema
or
Protobuf

stored in the source repository.

Later, if event contracts become more complex:

Schema Registry

can be introduced.

This follows the same cost principle: add complexity only when required.

---

11. Deduplication

Every event should contain a unique ID generated by the client:

event_id = UUID

This is important because web and mobile applications may retry event delivery.

The consumer can deduplicate events before inserting them into ClickHouse.

---

12. Dead Letter Queue

Invalid events should be moved to another Redpanda/Kafka topic.

For example:

events_dlq

This allows engineers to inspect and replay them later.

No event should be silently dropped.

---

13. Observability

The hybrid architecture should also keep monitoring simple.

I would use:

Prometheus
+
Grafana

Metrics should include:

API requests/sec

API error rate

Events received/sec

Queue lag

Consumer throughput

DLQ count

ClickHouse insertion latency

Data freshness

Disk usage

CPU

Memory

For logs, Loki can be added if centralized logging is necessary.

However, I would avoid deploying a large Elasticsearch cluster unless the team really needs it.

Again, the goal is to keep the architecture operationally lightweight.

---

14. Data Observability

Infrastructure monitoring alone is not enough.

We also need to monitor the data itself.

Useful metrics include:

Number of events per minute

Events by platform

Events by event_type

Duplicate percentage

Invalid event percentage

Latest event timestamp

Missing fields

Unexpected schema changes

For example:

Normal traffic:
200,000 events/hour

Current traffic:
15,000 events/hour

Even if all servers are healthy, this may indicate that the mobile SDK stopped sending events.

Therefore, data volume anomaly alerts are important.

---

15. Deployment

The initial deployment could be very small.

For example:

VPS 1
Tracking API
Nginx

VPS 2
Redpanda

VPS 3
ClickHouse

VPS 4
Monitoring

Optional second replicas

Historical data would be stored externally in cloud object storage.

Docker Compose can initially be used rather than Kubernetes.

This is another important cost decision.

I would **not introduce Kubernetes initially** unless:

- The company already uses Kubernetes
- Many services need orchestration
- Automatic scheduling and scaling are required
- The infrastructure team is already experienced with it

Otherwise Kubernetes introduces additional operational complexity without much benefit at this scale.

Scalability Strategy

The hybrid architecture can scale gradually.

Stage 1

Single API VPS
Single Redpanda node
Single ClickHouse node
Cloud Object Storage

Stage 2

When traffic increases:

Multiple API instances
3-node Redpanda cluster
ClickHouse replication

Stage 3

At much larger scale:

Kubernetes
Managed Kafka / MSK

Managed ClickHouse
Managed Flink

This architecture avoids paying enterprise-scale infrastructure costs before they are necessary.

Cost Comparison

Category	Self-Hosted	Fully Cloud	Hybrid
Infrastructure cost	Low	Medium / High	**Low**
Maintenance effort	High	Low	**Low / Medium**
Operational complexity	High	Low	**Low / Medium**
Scaling flexibility	Medium	High	**High**
Vendor lock-in	Low	High	**Low**
Historical storage cost	Medium	Low	**Very Low**
Initial setup complexity	High	Low	**Medium**
Suitable for small DE team	Medium	High	**High**
Cost efficiency overall	Medium	Medium	**High**

Cost-Efficiency Principles

The architecture is designed around several principles.

1. Avoid unnecessary distributed systems

Do not deploy:

Kafka + Flink + Kubernetes + Spark

simply because they are common Data Engineering technologies.

Every additional distributed system increases:

Infrastructure cost
+
Monitoring cost
+
Maintenance cost
+
On-call burden
+
Debugging complexity

Use them only when the workload requires them.

---

2. Use managed services where they are cheap

Object storage is a good example.

Running MinIO may appear cheaper, but the company must manage:

Storage capacity
Replication
Backups
Disk failures
Networking
Availability

Low-cost cloud object storage eliminates most of this work.

---

3. Self-host expensive compute services

ClickHouse is relatively easy to self-host and can run efficiently on VPS infrastructure.

This can provide significant savings compared with keeping a large analytical warehouse running continuously.

---

4. Keep raw data cheap

Raw event data should primarily live in compressed Parquet on object storage.

The analytics database should contain only data that needs fast access.

---

5. Scale incrementally

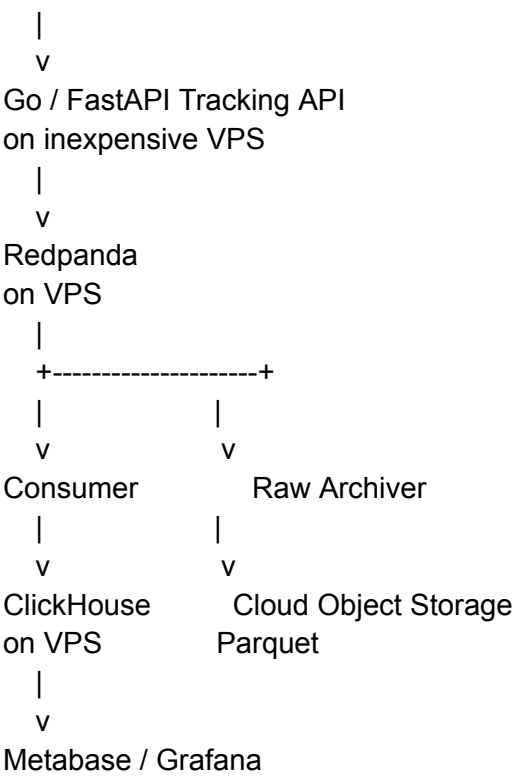
Infrastructure should grow with actual traffic.

The company should not provision for hypothetical future scale.

Final Recommendation

Given the requirements and the priority of cost efficiency, I would choose the **Hybrid Architecture**.

The recommended stack would be:



Monitoring:

Prometheus
+
Grafana

Data quality:

Schema validation
Deduplication
DLQ
Data freshness checks
Volume anomaly detection

The main reason I prefer this approach is that it optimizes both types of cost.

Infrastructure Cost

Expensive continuously running services such as the analytical database can run on relatively cheap VPS infrastructure.

Large amounts of historical data are stored in inexpensive object storage.

Engineering / Maintenance Cost

The architecture intentionally avoids unnecessary components such as:

Kubernetes
Flink
Spark
Large Elasticsearch clusters
Complex orchestration platforms

unless the workload grows enough to justify them.

Cloud services are used selectively where they significantly reduce operational burden, especially durable object storage and edge infrastructure.

Therefore, the architecture provides a practical balance:

Low infrastructure cost
+
Low operational complexity
+
Easy future scaling
+
Near real-time analytics
+

Cheap historical storage

For the current requirement of millions of events per day and processing within minutes, this architecture should provide enough capacity without over-engineering the system.